MANIFESTO.md

The OMI Manifesto

Notation as Cipher. Cipher as Notation.

OMI begins with a simple claim:

> Computation is not the mutation of data.
> Computation is the lawful transformation of interpretation.

The binary source remains.

The reading changes.

The rewrite is the computation.

---

1. The Third Collapse

Every major computing paradigm begins by collapsing a boundary that once seemed natural.

Lisp collapsed program and object.

Lisp showed that code and data could share one form.

```
code ↔ data
```

An S-expression can be read as structure, stored as data, or evaluated as program.

POSIX collapsed storage and communication.

POSIX showed that files, pipes, sockets, and devices could share one interface.

```
storage ↔ communication
```

A file descriptor can point to a disk file, a terminal, a pipe, or a network socket.

OMI collapses representation and interpretation.

OMI shows that notation and cipher can share one surface.

```
representation ↔ interpretation
notation ↔ cipher
```

The same binary source can be read as text, address, proof, graph, route, receipt, or rewrite table depending on the declared interpretation.

This is the OMI collapse.

---

2. OMI Is Not a Database

OMI is not a database.

A database says:

```
address → stored value
```

OMI says:

```
source of truth → rewrite surface → routed interpretation → receipt
```

OMI does not begin with stored data.

OMI begins with a versioned binary source of truth.

That source is loaded as a rewrite table.

The table is not meaningful by itself.

Meaning appears only when a lawful notation-cipher is declared.

---

3. OMI Is a Rewrite Register

OMI is better understood as a rewrite register.

A rewrite register does not ask:

```
What value is stored here?
```

It asks:

```
What lawful transformation is declared here?
```

The primitive object is not a row in a database.

The primitive object is a transition.

The unit is not data.

The unit is a bitblip: a small binary transition inside a rotating rewrite surface.

---

4. Notation as Cipher

A cipher is usually understood as a method for hiding meaning.

OMI uses cipher differently.

In OMI, a cipher does not hide meaning.

A cipher declares meaning.

A notation is a cipher because it tells the reader how to interpret the same surface.

A cipher is notation because it defines the rules by which a surface becomes readable.

```
notation as cipher
cipher as notation
```

This is not secrecy.

This is interpretation.

---

5. The Binary Source Remains

In ordinary computation, we say the data changes.

In OMI, the source may remain unchanged.

What changes is the reading.

The same binary surface may be interpreted as:

```
text
address
instruction
proof
receipt
graph
route
nomogram
rewrite table
```

The bits remain.

The interpretation rotates.

---

6. The Omicron Selector

The Omicron is not merely a symbol.

It is an interpretation selector.

```
omi-
```

and

```
-imo
```

are dual readings of the same structure.

One opens the address perspective.

One closes or mirrors the interpretation perspective.

The Omicron functions like a sign bit, but not as positive or negative.

It selects the active notation.

It declares how the source is to be read.

---

7. The Slash Path

The identity surface is:

```
omi---imo
```

The routed interpretation path is:

```
/---/
```

The slash path does not store data.

It declares how the source is read.

A fully qualified OMI expression has the shape:

```
omi-<frame>/<control>/<scale>/<relation>/<unit>-imo
```

The source remains stable.

The path declares the route.

---

8. Control Codes Are Rewrite Operators

The ASCII control range:

```
0x00..0x1F
```

is not merely a set of non-printing characters.

These codes were historically protocol operations:

```
NUL
SOH
STX
ETX
EOT
ENQ
ACK
```

```
BEL
BS
HT
LF
CR
SO
SI
```

They describe transitions.

They start, stop, shift, acknowledge, separate, return, and advance.

OMI restores their original force.

Control codes are not characters.

They are rewrite operators.

---

9. Printable Characters Are Projections

The printable range:

```
0x20..0x7F
```

is the visible projection of a rewrite surface.

The space character:

```
0x20
```

is not nothing.

It is visible absence.

It is the printable empty pivot.

The delete character:

```
0x7F
```

is not ordinary text.

It is a terminal sentinel.

Together they mark a projective boundary between control and projection.

---

10. Rotation Instead of Shifting

OMI does not shift code space.

OMI rolls code space.

A shift loses information.

A rotation preserves it.

Nothing falls off the edge.

The edge returns through the orbit.

The canonical transition law is:

```
ΔC(x) = rotl(x,1) XOR rotl(x,3) XOR rotr(x,2) XOR C
```

The variable `C` is the carried closure of the orbit.

Every rewrite leaves a witness.

Every witness can carry forward.

---

11. The Projective Closure

A finite row is not merely a line.

When rolled, it becomes a projective closure.

For a 4-bit wheel:

```
2^4 = 16
```

OMI reads this as:

```
15 active states
1 closure state
```

This is the 15-of-16 principle.

For an 8-state wheel:

```
2^3 = 8
```

OMI reads this as:

```
7 active states
1 closure state
```

This is the 7-of-8 principle.

The missing state is not lost.

It is the projective center.

---

12. The Empty List Principle

Lisp reveals a deep computational truth:

```
()
```

is both an atom and a list.

OMI treats closure the same way.

The closure state is both:

```
a point
and
a space
```

It is origin and container.

It is empty and structural.

This is why OMI can begin with nothing and still define a lawful rewrite.

---

13. Unicode as Rewrite Space

Unicode is usually treated as a character set.

OMI treats it as a rewrite manifold.

A codepoint is not merely a glyph.

It is an incidence cell.

Rows, columns, blocks, sentinels, private-use regions, and direction markers define lawful interpretation surfaces.

Private-use regions are especially important because they give OMI bounded regions for protocol-defined notation without colliding with public character meanings.

---

14. Incidence, Not Storage

OMI reads character tables as incidence systems.

```
points  = codepoints
blocks  = rows, columns, mirrors, sentinels, escape sets
edges   = lawful traversals
receipt = accepted incidence proof
```

The table is not merely a lookup table.

It is a finite geometry of possible rewrites.

---

15. Omi-Nomogram

OMI uses runtime scales to declare how a relation is read.

This surface is called:

```
Omi-Nomogram
```

Its mechanical face is:

```
Omi-SlideRule
```

A nomogram does not store answers.

It aligns scales.

Likewise, Omi-Nomogram does not store meaning.

It declares the scale of interpretation.

Canonical scale row:

```
0x30 identity / unity
0x31 multiply-divide
0x32 square-root
0x33 cube-root
0x34 folded π scale
0x35 reciprocal scale
0x36 sine-cosine
0x37 tangent-cotangent
0x38 small-angle / degree-radian
0x39 Pythagorean
0x3A log10
0x3B natural log / exponential
0x3C sexagesimal gate
0x3D arbitrary powers
0x3E quadratic / gnomon
0x3F period / replay / LFSR scale
```

The scale declares the reading.

The receipt accepts the result.

---

16. BiDi as Dual Reading

BiDi is not merely text direction.

In OMI, BiDi expresses lawful dual interpretation.

The same source may be read:

```
forward
backward
mirrored
folded
projected
```

without changing the source.

This is why OMI treats direction as part of computation.

Direction is interpretation.

Interpretation is rewrite.

---

17. Receipts, Not Belief

OMI does not ask the reader to believe a projection.

OMI requires a receipt.

A projection may display.

A route may suggest.

A graph may navigate.

A reconstruction may produce a candidate.

But only a receipt accepts.

Canonical rule:

```
The reader may recognize.
The resolver may promote.
Only validation and receipt may accept.
```

---

18. The OMI Difference

Traditional systems say:

```
data is transformed by code
```

OMI says:

```
notation transforms interpretation
```

and therefore:

```
notation is cipher
cipher is notation
```

The source remains.

The reading changes.

The rewrite is the computation.

---

19. The Three Doctrines

```
Lisp:
code ↔ data

POSIX:
file ↔ device

OMI:
notation ↔ cipher
```

Lisp collapses program and object.

POSIX collapses storage and communication.

OMI collapses representation and interpretation.

---

20. The Manifesto

OMI is not a database.

OMI is not a file system.

OMI is not merely a protocol.

OMI is a rewrite geometry of computation.

It preserves versioned binary sources of truth.

It routes them through declared notation.

It interprets them through lawful scales.

It rotates them through projective closure.

It validates them through receipts.

The binary source remains.

The interpretation changes.

The computation is the rewrite.

One-Line Canon

```
Lisp collapsed program and object. POSIX collapsed storage and communication.
OMI collapses representation and interpretation: the same binary surface is both
notation and cipher, distinguished only by the active reading, because
computation is the rewrite of interpretation, not the mutation of data.
```

Shortest Form

```
OMI is notation as cipher and cipher as notation.
```